

An Edge Node for Citizen-Contributed Maritime Observations

Carmine Coppola, Vincenzo Bucciero, Simone Perrotta, and Raffaele Montella
Department of Science and Technology, University of Naples “Parthenope”, Naples, Italy
{carmine.coppola001, vincenzo.bucciero001, simone.perrotta001}@studenti.uniparthenope.it
raffaele.montella@uniparthenope.it

Abstract—Citizen-contributed observations can complement conventional maritime monitoring, but their scientific value depends on reliable acquisition, explicit provenance, reproducible processing, and responsible data governance. This paper presents a modular edge node designed for future citizen-observatory deployments on small vessels and coastal sites. The platform separates hardware access, source supervision, frame delivery, inference, normalized detections, events, sessions, persistence, and operator interaction through stable interfaces. Live and replay inputs are exposed through common downstream interfaces, enabling controlled validation of the observation workflow. A Raspberry Pi 4 prototype integrates a dual-view RGB camera assembly, an on-demand FLIR thermal sensor, ONNX inference, REST services, and a browser dashboard. All 310 automated tests passed; scripted hardware and browser checks complemented them. During 151 steady-state observations, both logical RGB feeds maintained 10 FPS while the platform process tree reported 53.16% mean CPU utilization (with 100% denoting one logical core) and approximately 450 MiB of resident memory. Fifty replay-based inference requests required 1.08 s mean end-to-end latency (0.94 s median); stage-level instrumentation guided changes that reduced latency from 3.32 s to 0.91 s, approximately 72%, without changing the inference backend. A synthetic thermal-stress test peaked at 77.4°C against a 78°C cutoff without throttling, and a two-hour continuous-inference run completed 2530 requests without error, although request latency under sustained load more than doubled. Beyond implementation, the paper defines a node-to-observatory contract covering participant roles, observation metadata, consent-aware retention, quality control, controlled dissemination, and contributor feedback. The contribution is a validated technical foundation and deployment blueprint for a subsequent multi-node participatory study, not a claim of a completed participatory deployment, fused multimodal perception, or safety certification.

Index Terms—citizen science, citizen observatory, participatory sensing, maritime monitoring, edge computing, computer vision, provenance, reproducibility

I. Introduction

Maritime and coastal environments are observed through authoritative systems such as fixed stations, survey vessels, remote sensing, and regulated vessel-reporting infrastructures. These systems are indispensable, yet their spatial and temporal coverage is constrained by cost, deployment opportunities, and operational priorities. Citizen science offers a complementary pathway: recreational sailors, fishers, coastal communities, and professional operators can contribute observations acquired during ordinary activities, extending coverage while strengthening

public participation in environmental research [1], [2]. In this work, crowdsourcing denotes the voluntary and distributed contribution of maritime observations by vessel operators and coastal observers, subject to provenance, quality-control, and governance requirements. Its value is not created by scale alone: it depends on traceable acquisition, suitable metadata, transparent processing, quality assurance, privacy safeguards, and feedback that lets contributors understand how their effort supports scientific or societal objectives [3]–[5].

Maritime vision is demanding for participatory sensing: reflections, glare, waves, fog, rain, platform motion, and long viewing distances complicate observation of people, small craft, buoys, and floating objects. RGB and thermal sensors can provide complementary evidence, while edge processing can reduce data transmission and preserve local operation under intermittent connectivity. A detector alone is not a citizen-science infrastructure; a field-ready node must coordinate heterogeneous sources, maintain explicit state, retain provenance, support repeatable processing, and remain replaceable at the sensor and model level.

This paper presents a modular edge node for citizen-contributed maritime observations. The work addresses a systems question that precedes a large participatory campaign: Can a low-cost embedded node provide a reproducible and governable observation path without coupling acquisition, inference, persistence, and presentation? We study three research questions:

- RQ1: Can heterogeneous live and recorded sources be processed through one model-independent workflow suitable for repeatable validation?
- RQ2: Can the integrated software stack operate within the resource envelope of a Raspberry Pi 4 while exposing measurable end-to-end behavior?
- RQ3: Which technical and governance interfaces are required to elevate a standalone edge prototype into a responsible citizen-observatory node?

The contributions are: (i) a modular architecture separating sensing, source management, frame delivery, inference, detections, events, sessions, persistence, and visualization; (ii) a common live/replay path and normalized observation representation; (iii) an empirical characteri-

zation of the integrated prototype; and (iv) a node-to-observatory contract that distinguishes implemented capabilities from the federation, governance, and engagement mechanisms to be evaluated in the next field phase.

II. Background and Positioning

A. Citizen Observatories and Data Quality

Citizen science can expand scientific capacity and public engagement, but projects must be designed around explicit scientific questions, participant roles, feedback, and data stewardship [1], [6]. Citizen observatories emphasize community participation, environmental sensing, digital platforms, and reciprocal communication [4]; marine initiatives show that opportunistic vessel measurements can increase ocean-observation density when instrumentation, metadata, and validation are handled systematically [2].

Data quality in citizen science is multidimensional: accuracy, completeness, consistency, provenance, protocol adherence, and fitness for a declared purpose, combining prevention (training, constrained interfaces) with correction and assessment (diagnostics, duplicate detection, expert review) [3]. The proposed platform concentrates on the technical node layer: acquiring and processing observations consistently and retaining the context required for later assessment. Recruitment, co-design, and societal impact remain field-study concerns rather than properties inferable from a laboratory prototype.

B. Maritime Vision and Edge Processing

Maritime object detection has been studied for navigation, surveillance, and search and rescue. SeaDronesSee demonstrates the difficulty of locating people and small objects in open-water imagery under changing altitude, scale, illumination, and sea state [7]; related work reports similar challenges for small floating objects and maritime target recognition [8], [9]. Thermal imagery may retain contrast under low illumination but requires dedicated calibration and does not by itself constitute sensor fusion.

Edge computing can reduce response time and network dependence by moving processing near the source [10], but model execution is only one component of end-to-end behavior: acquisition, format conversion, preview generation, event handling, persistence, and API construction can materially affect latency and energy use. The proposed work therefore contributes an instrumented application architecture rather than a new detector.

C. Scope and Novelty

The prototype supports heterogeneous visual sources but does not yet synchronize or geometrically register RGB and thermal frames, so we use heterogeneous sensing, not multimodal fusion; it is also not a regulated distress, navigation, or surveillance system. Its novelty is the explicit connection between a modular embedded perception workflow and the provenance, governance, and feedback obligations of a citizen observatory, expressed as a node-to-observatory contract and evaluated at the node level.

D. Research Gap and Design Implications

The literature establishes citizen participation, maritime perception difficulty, and edge-processing benefits, but these strands are commonly treated in isolation: citizen-science platforms often assume mobile-form or cloud-connected instruments, while embedded-vision prototypes often optimize a fixed sensor-model pipeline without retaining context for scientific reuse. A maritime citizen observatory needs both: an edge node autonomous under intermittent connectivity, producing records that can later be interpreted and compared across contributors and deployments.

This gap leads to four implications: the observation, not the image or detection alone, is the primary scientific object; replay must be operationally equivalent to live acquisition; provenance and component state must be generated during execution, not reconstructed afterward; and federation must be a governed exchange of selected records, not unconditional raw-imagery streaming.

E. Positioning Against Prior Work

Maritime datasets and embedded-vision studies mainly evaluate perception performance, whereas citizen-oceanography and citizen-observatory initiatives emphasize participation, stewardship, and scientific reuse [2], [4], [5], [7]–[9]. EASY connects these perspectives through edge execution, a common live/replay path, modular boundaries, explicit provenance, and a governance interface for future federation; the participatory component is a deployment objective, not presented here as experimentally validated.

III. System Architecture

A. Design Principles

The platform follows five principles. First, hardware-specific operations remain isolated from processing logic, so heterogeneous sensors can be substituted without restructuring inference code. Second, live and recorded sources expose the same downstream interface, so protocol and detector changes can be validated on retained replay data before they reach contributors. Third, model-specific outputs are converted into normalized detections, keeping the observation record meaningful independently of any particular detector. Fourth, component state and health are explicit, distinguishing application readiness from sensor availability rather than one opaque status. Fifth, each stage is measurable and tied to source and session context, so latency and failures can be attributed to a specific stage rather than an undifferentiated end-to-end figure.

B. Components and Execution Path

Figure 1 shows the principal components. The System Orchestrator initializes services, resolves dependencies,

EDGE NODE - SOFTWARE ARCHITECTURE

Embedded Maritime Situational Awareness

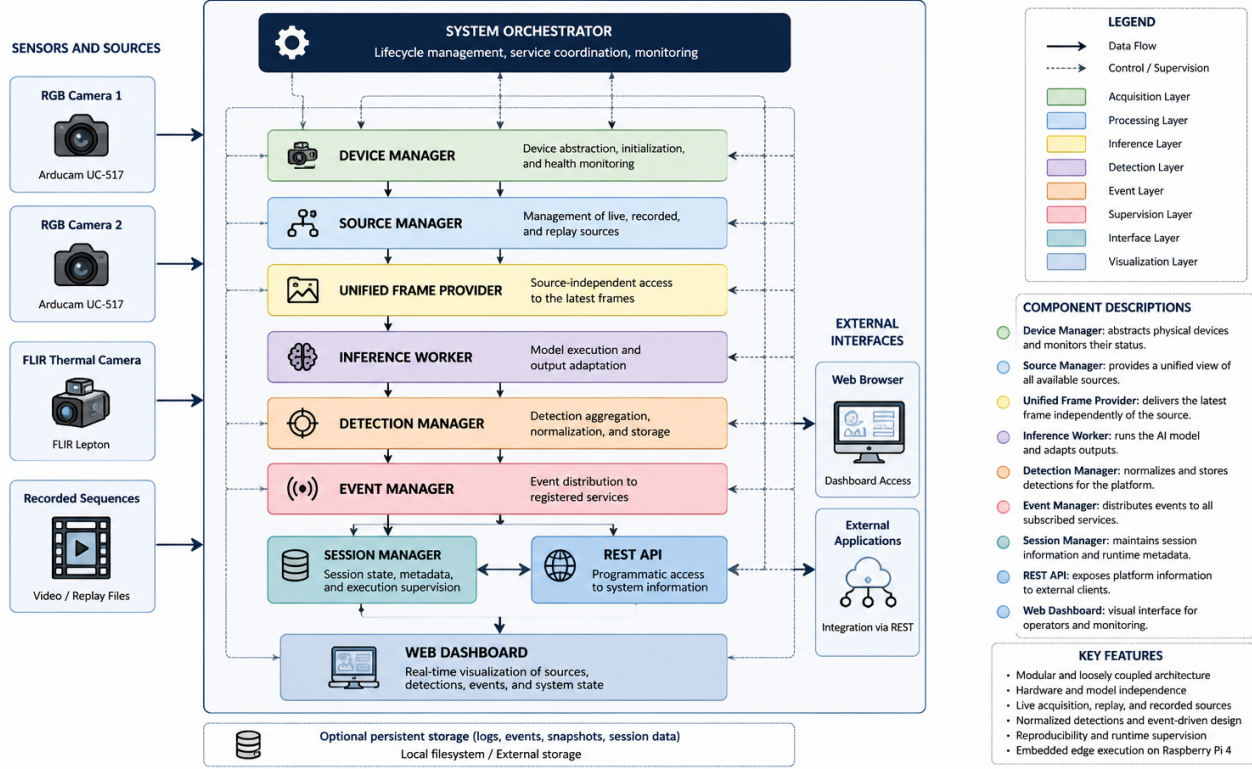


Fig. 1. Software architecture of the proposed edge node. The two RGB blocks denote physical views delivered through one shared acquisition stream; services communicate through explicit boundaries.

and aggregates runtime state. The Device Manager represents physical devices independently from source activation. The Source Manager exposes live cameras, stored video, and replay sessions through a common abstraction. The Unified Frame Provider returns the latest valid frame and source context without exposing device-specific acquisition details. The Inference Worker performs preprocessing, executes the configured ONNX model, and adapts backend-specific outputs to a common detection form. The Detection Manager distributes normalized results. The Event Manager propagates state and application events, while the Session Manager associates observations, results, and operator actions with an execution context. REST services and the browser dashboard consume these coordinated platform services rather than camera or model APIs directly.

The orchestrator initializes services in dependency order and distinguishes application readiness from sensing availability. This distinction is important in field operation: a node may remain usable when one sensor is unavailable, while an on-demand device may be ready without being continuously active. Device state is therefore represented separately from source activity. External clients interact with coordinated services instead of accessing cameras or model code directly, preserving the same boundaries for

browser, automation, and future federation clients.

C. Observation Record and Provenance

A contribution is represented as an observation record whose identity is independent of any particular detector. The current implementation retains source, session, capture and processing context, detections, events, and persisted artifacts. The deployment contract extends this representation with contributor-controlled and federation-oriented metadata. Table I separates mandatory node-level fields from fields that become meaningful only in a governed multi-node study: an entry marked “node” is populated automatically by the running prototype from local execution, with no human or federation step involved, whereas an entry marked “deployment” requires a project-level policy decision or a multi-node federation service that does not yet exist in this prototype.

The record deliberately separates raw imagery, derived detections, and descriptive metadata: a deployment may retain only a preview and derived record, keep full imagery locally until review, or submit selected imagery to a controlled repository, supporting data minimization while preserving context for later assessment. Model scores remain observations from a configured algorithm, not

TABLE I

Observation record proposed for scientific exchange. Fields marked “node” are produced by the current execution path; “deployment” fields depend on the participatory protocol or federation service.

Group	Representative fields	Origin	Purpose
Identity	observation, source, session, and sequence identifiers	node	stable association among frames, detections, events, and persisted products
Time	capture time, processing time, stage durations, and replay position	node	ordering, latency analysis, and repeatable execution
Acquisition	source type, device state, dimensions, frame rate, and modality	node	interpretation of heterogeneous inputs and device diagnostics
Processing	model identifier, configuration, confidence, class, and bounding region	node	traceable derivation without exposing backend-specific tensors
Contribution	node enrollment, contributor pseudonym, protocol version, and consent state	deployment	governance, attribution, withdrawal, and protocol compliance
Spatial context	optional position, uncertainty, sensitivity class, and generalization policy	deployment	scientific use while limiting disclosure of sensitive trajectories
Quality	completeness, diagnostics, duplicate flags, cross-observation agreement, and curator status	mixed	fitness assessment and transparent filtering
Dissemination	retention class, access level, license, embargo, and feedback status	deployment	controlled reuse and reciprocal communication

calibrated probabilities or verified events, unless independently validated.

Acquisition and inference are decoupled: cameras continue publishing current frames while selected frames are processed on request; replay sources traverse the same frame provider, inference, event, persistence, and visualization path as live sources, providing the basis for repeatable validation and regression testing.

D. Observation Contract and Implementation Status

A citizen-science contribution is more than a bounding box. The platform therefore defines an observation contract containing source and session identity, capture and processing time, model context, result confidence, spatial extent, and device state. Optional location and contributor identifiers are governed by project policy rather than assumed to be public. Table II distinguishes the capabilities exercised in the prototype from those reserved for a federated deployment.

Source, session, event, timing, local persistence, export, and export-retention foundations are implemented and tested; contributor-controlled disclosure, cryptographic signing, remote federation, cross-node quality analysis, and participant-facing feedback are next-phase requirements, not completed results.

IV. Participatory Deployment Blueprint

A. Roles and Contribution Lifecycle

The deployment blueprint separates responsibilities often conflated in technology-centric prototypes: a contributor operates a node and selects observations to share; a project coordinator defines protocols, consent terms, and retention periods; a scientific curator assesses quality

TABLE II
Node-to-observatory contract and current status.

Concern	Contract	Status
Provenance	source, session, capture/process time, model context, device state	implemented locally
Repeatability	common live/replay path and stage timing	implemented
Data minimization	local retention, export, and retention controls	partial local implementation
Privacy	consent, sensitive locations, imagery filtering	deployment policy
Quality control	diagnostics, confidence, duplicates, cross-node agreement, review	partial
Federation	enrollment, signing, secure sync, metadata export	planned
Feedback	maps, summaries, explanations, contribution outcomes	planned field study

and constructs research datasets; an authorized maritime stakeholder may consume validated information, but automated outputs remain advisory.

The intended lifecycle comprises seven stages: enrollment; local acquisition; edge processing; optional contributor review; selective submission with provenance;

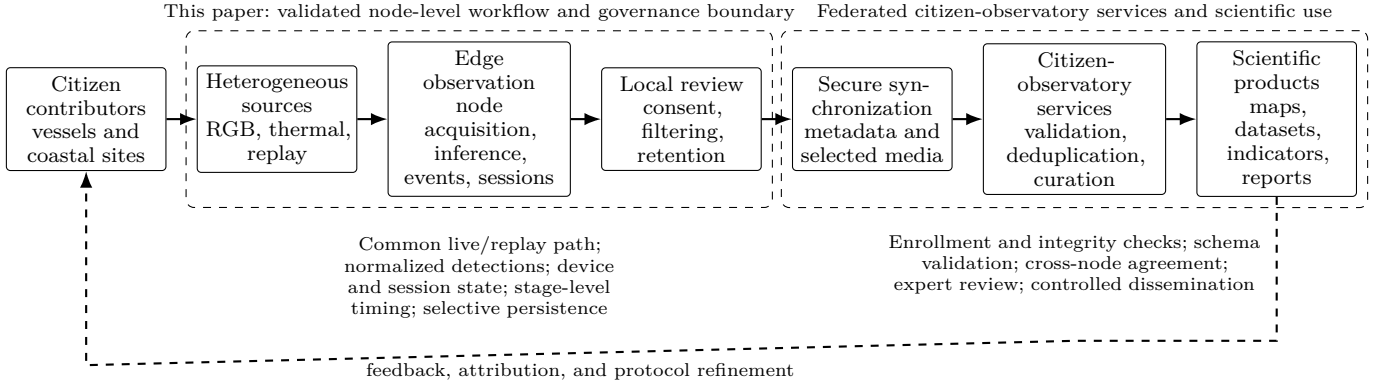


Fig. 2. Citizen-contributed maritime observation flow. This paper validates the edge-node segment and defines its interface with a future federated citizen observatory.

automated and expert quality assessment; and feedback through maps, summaries, and outcome explanations, treating citizens as informed collaborators rather than passive sensor carriers.

B. Quality, Privacy, and Responsible Use

Quality control should combine protocol constraints, device diagnostics, duplicate detection, separately calibrated model outputs, cross-observation agreement, and expert review; a model score must not be read as calibrated certainty unless separately validated. Sensitive imagery may reveal identifiable persons or location histories, so privacy-preserving defaults should minimize collection, separate public from controlled-access products, and support local deletion before transfer.

The blueprint also defines a boundary of use: the system is an environmental observation platform, not a substitute for regulated distress, navigation, or vessel-traffic systems. A participatory field study must evaluate usability, consent comprehension, retention, and contributor-feedback effectiveness; these outcomes cannot be established from technical tests alone.

C. Federation and Quality Workflow

Federation is designed as an asynchronous exchange because coastal and small-vessel connectivity can be intermittent. A node creates a locally complete record, applies the contributor’s retention and disclosure choices, and queues eligible records for outbound transfer; the receiving service verifies enrollment and integrity, checks compatibility, and assigns a curation state, while automated checks flag missing metadata, inconsistent timestamps, or near-duplicate observations, and cross-node agreement or expert review can then adjust confidence.

This workflow keeps three notions distinct: model confidence (from the detector), record quality (completeness, provenance, diagnostics), and scientific acceptance (a project-specific curation decision). A rejected record can still improve protocols or document adverse acquisition conditions.

D. Representative Deployment Scenarios

The architecture supports three complementary scenarios: an opportunistic vessel node, where a contributor records and reviews observations during ordinary navigation; a fixed coastal node, maintaining a stable viewpoint under a site-specific retention policy; and a curated campaign, where coordinators distribute a common protocol and replay package to compare configurations across devices before field collection. These scenarios share the same node-level execution path while differing in consent, connectivity, and quality-control policy.

V. Prototype and Evaluation

A. Configuration and Method

The prototype ran on a Raspberry Pi 4 with 8 GB memory, 64-bit Linux 6.1.21-v8+, and Python 3.9.2, with dependency versions pinned in the artifact repository. Its RGB subsystem used an Arducam stereo camera assembly exposed by Linux as one IMX477 device. EASY acquired one 1280×480 MJPEG stream at 10 FPS and split it into left and right logical views. A FLIR thermal module was accessed on demand. Flask exposed REST services, ONNX Runtime executed the detector on the CPU, and psutil provided the resource instrumentation reported below. The detector served as a reproducible processing workload; detector accuracy was outside the experimental scope. RGB and thermal frames were neither hardware-synchronized nor geometrically registered.

The evaluation addresses RQ1 and RQ2 through five complementary experiments (Table III). Automated regression tests examined component contracts; scripted Raspberry checks exercised hardware workflows; and browser checks covered operator views. Steady-state monitoring characterized acquisition and supervision. Replay-based requests isolated the downstream inference path under repeatable input. REST and startup measurements characterized operator-facing responsiveness. A synthetic thermal-stress experiment and a separate two-hour continuous-inference run addressed reviewer requests for

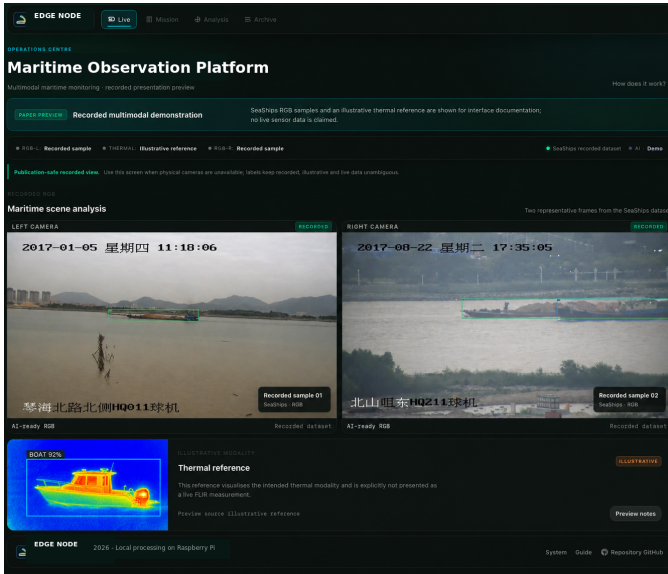


Fig. 3. Publication-safe dashboard preview with recorded SeaShips RGB samples [11] and an explicitly illustrative thermal reference. It documents the operator interface and does not represent simultaneous live-sensor acquisition.

TABLE III
Evaluation protocol.

Experiment	Protocol
Functional validation	310 automated tests (150 service, 160 interface), plus scripted Raspberry and browser checks
Steady state	151 observations with both logical RGB feeds active at 10 FPS
Inference timing	50 successful replay-based requests with stage-level timing
REST latency	50 requests per selected endpoint; three service-start readiness runs
Thermal robustness	120 s synthetic full-core CPU load within the same run; separate 2-hour continuous-inference run

evidence beyond a single lab-condition snapshot. Reported percentiles are descriptive because the inference sample contains fifty requests; no population-level latency claim is made.

B. Instrumentation and Measurement Boundaries

Measurements were collected from the integrated application rather than from an isolated inference script. Process-tree CPU uses 100% for one fully occupied logical core; system CPU is aggregated across four cores. Resident-set size is summed across the platform process tree and may count shared pages more than once. Source frame rate and component state were sampled with resource values so that low utilization could not be mistaken for successful acquisition when a stream had silently stopped. Startup was measured from systemctl start until the readiness

resource reported the orchestrator as running; Raspberry Pi boot time was excluded.

The inference path was decomposed into acquisition, preprocessing, backend execution, postprocessing, preview generation, persistence, and request-pipeline intervals. End-to-end API latency was measured separately, allowing uninstrumented framework overhead to be estimated from the difference. Replay provided a repeatable input through the same frame-provider and downstream service interfaces used by live sources. Percentiles use the nearest-rank method; with fifty inference requests, P95 is the 48th-ranked observation rather than the sample maximum, but it remains a descriptive single-device statistic and is not presented as a population-level tail-latency estimate.

C. Functional Results

All 310 automated tests passed (150 service-level, 160 interface-level). Scripted Raspberry validation and browser checks additionally covered device registration, source activation, dual-view RGB acquisition, on-demand thermal capture, replay, source-independent frame delivery, ONNX inference, normalized detections, event propagation, persistence, session updates, and presentation. Both logical RGB feeds delivered real frames at 10 FPS. The thermal source acquired a real frame and advanced its sequence state. Live acquisition and replay-based inference were exercised separately, while contract tests confirmed that both source types enter the same frame-provider and downstream service interfaces. This evidence supports RQ1 without claiming simultaneous RGB-thermal processing.

The tests establish internal functional consistency, not detector accuracy, field robustness, or safety performance. No precision, recall, average precision, or distress-detection claim is inferred from these experiments. Two auxiliary observations about the configured detector are nevertheless relevant to this paper's argument about provenance, and are reported as context rather than as contributions. First, a sequence-level audit found that an earlier training split placed frames from the same video sequence in more than one subset: 11 of 36 sequences were affected and the reported mAP50 of 0.942 was therefore optimistic; rebuilding the split so that each sequence belongs to exactly one subset reduced the same model to 0.382, and adding a further public maritime dataset under the corrected split recovered 0.627. Second, on public near-shore video never used in training, a diagnostic transfer pilot measured 1.69% vessel AP50 for the frozen model. The measurement procedure did not change in either case; the provenance of the data did. This is precisely the failure mode that the observation record is designed to make visible, and it motivates the field acquisition campaign described in Section VII.

TABLE IV
Integrated functions exercised during validation.

Function group	Observed outcome
Initialization and readiness	coordinated services reached ready state
Dual-view RGB acquisition	both logical feeds sustained configured delivery
Thermal acquisition	on-demand frame and sequence state updated
Live/replay contract	common frame-provider and downstream interfaces exercised
Inference and detections	ONNX execution and normalized records completed
Events, sessions, persistence	associations retained across the request path
REST and dashboard	state, controls, diagnostics, desktop and mobile views exercised

TABLE V
Steady-state resource utilization (151 observations).

Metric	Mean	Median	P95	Max
System CPU (% , aggregate)	17.84	15.80	29.30	46.70
Process CPU (% , one core)	53.16	51.50	65.50	154.49
Process-tree RSS (MiB)	450.36	459.69	480.02	486.34
CPU temp. (°C)	56.48	56.48	57.45	58.91
Left RGB (FPS)	10.00	10.00	10.00	10.00
Right RGB (FPS)	10.00	10.00	10.00	10.00

D. Resource Utilization

Table V reports 151 steady-state observations. The platform used 53.16% of one logical core and 450.36 MiB summed process-tree RSS on average. Both RGB feeds maintained 10 FPS without component-state transitions. Mean CPU temperature was 56.48°C and the maximum was 58.91°C. This phase included acquisition, supervision, REST availability, and dashboard support, but excluded continuous inference and repeated thermal capture. Both figures are higher than in an earlier revision of this measurement (29.97% and 291.71 MiB) because the integrated codebase grew: local authentication, an indexed snapshot store, a rebuilt operator interface, and a detector retrained on an additional public dataset. The process-tree accounting method is unchanged, and the envelope remains a small fraction of a four-core, 8 GB board.

Two further experiments target the resource envelope directly. A synthetic full-core CPU load sustained for 120 s (mean system CPU 92.54%) triggered no throttling or under-voltage event, read from the same firmware interface (`vcgencmd get_throttled`), but reached 77.42°C against the platform’s 78°C safety cutoff, leaving only 0.6°C of margin; both RGB feeds continued to deliver 10 FPS throughout. Passive cooling is therefore a precondition for enclosed or warm deployments. A separate two-hour continuous-inference run completed 2530 requests with no crash, no unrecovered error, and no throttling; temperature stabilized at 64–68°C and resident memory grew from 521 to 529 MiB over the two hours, consistent with

TABLE VI
Replay-based request timing (50 requests).

Stage	Mean (ms)	Median	P95
Frame acquisition	60.10	57.52	85.98
Preprocessing	49.97	46.62	63.48
Inference backend	588.44	551.16	782.07
Postprocessing	2.46	1.55	3.55
Preview generation	84.06	70.51	210.54
Persistence	290.02	187.50	851.26
Request pipeline	1083.77	937.82	1675.30

bounded in-memory session history. Under that sustained back-to-back load, however, mean request latency rose to 2.34 s (median 2.21 s), roughly twice the isolated request-oriented figure reported in Section ??: the request-level numbers below characterize on-demand observation, not a saturated duty cycle. Neither measures electrical power in Watts, which needs external instrumentation and remains future work (Section VII), but both provide measured thermal and operational margin under sustained load rather than an untested assumption.

A third experiment addresses the acquisition path rather than the request path. The evaluated service captures thermal frames on demand; a candidate persistent thermal worker, kept behind a feature flag and not enabled on the deployed unit, was exercised for two hours alongside both RGB views. It produced 64,801 valid Y16 frames at 9.11 FPS while the two logical RGB views delivered 68,754 and 68,766 frames at approximately 9.55 FPS each, with no invalid frame, no stream restart, a raw ring buffer that stayed within its three-frame bound, and CPU temperature between 56.0 and 63.3°C; the installed service remained ready throughout. This establishes that the CSI and USB acquisition paths coexist on this hardware. It does not establish synchronized exposure, geometric registration, radiometric calibration, or multimodal fusion, which remain separate capabilities with their own validation requirements.

E. End-to-End Timing

Fifty replay-based inference requests completed successfully. Mean backend execution was 588.44 ms, while the complete instrumented pipeline required 1083.77 ms on average and 937.82 ms at the median (Table VI). The mean is pulled up by a small number of slow persistence writes, so both statistics are reported. Frame acquisition, preprocessing, preview generation, and persistence together accounted for a substantial portion of the request. Mean end-to-end API latency was 1088.89 ms, leaving approximately 5 ms outside the instrumented stages.

Compared with the baseline implementation, mean end-to-end latency in the optimized implementation decreased from 3.32 s to 0.91 s over fifty requests, or approximately 72%. Persistence was first isolated after stage instrumentation was introduced, when it averaged 1.77 s; after optimization it averaged 140 ms. Because stage instrumentation postdated the baseline measurement, the 1.77-s value is not a decomposition of the 3.32-s baseline.

TABLE VII
REST latency over 50 requests per endpoint.

Endpoint	Mean (ms)	Median	P95
/api/inference/status	6.45	6.00	10.52
/api/stream-state	5.49	4.90	7.93
/api/system/components	291.32	273.39	361.32
/health	208.60	205.94	221.53

Concretely, the baseline issued one synchronous JSON write per detected object and a matching per-object event-log update, so a multi-detection frame triggered several redundant persistence round-trips; the optimized implementation batches all detections and events from one inference frame into a single write each, regardless of object count, removing the redundant round-trips without changing the storage format, adding asynchronous I/O, or altering the inference backend. The measurements identified avoidable software overhead that model-only timing would have obscured.

During the inference phase, mean system CPU was 40.62%, process-tree CPU was 145.18% (about 1.45 logical cores), process-tree RSS was 496.86 MiB, and CPU temperature averaged 65.52°C (66.70°C maximum). Both logical RGB feeds continued to report 10 FPS. Lightweight inference-status and stream-state endpoints responded in 14.65 and 6.12 ms on average; component and comprehensive health endpoints required 719.25 and 275.94 ms over fifty requests each. Application readiness averaged 10.9 s over three controlled restarts, up from 6.52 s, because more services now start at boot. These results answer RQ2 for request-oriented observation and supervision, while ruling out any claim of continuous 10-FPS inference.

The latency gradient reflects the amount of state assembled by each resource. Stream and inference status are suitable for frequent dashboard refresh, whereas the comprehensive health representation is better suited to periodic diagnostics. This separation prevents an operator interface from repeatedly triggering the most expensive system-wide inspection and provides a basis for future remote monitoring under constrained connectivity.

VI. Discussion

A. Answers to the Research Questions

RQ1 is supported by separate live-acquisition and replay-inference trials, together with contract tests of their shared frame-provider and downstream interfaces. RQ2 is supported for the evaluated operating mode: dual-view acquisition and supervision fit within the Raspberry Pi 4 memory and CPU envelope, each on-demand replay inference request requires approximately 1.1 s on average and 0.9 s at the median, and the envelope also held under synthetic thermal stress and two hours of continuous inference, with the two qualifications recorded in Section V-D: a 0.6°C thermal margin under full load, and roughly doubled latency under a saturated duty cycle. RQ3 is answered only at the design level, deliberately so: the

proposed observation record, contract, and deployment blueprint define the technical and governance interfaces a responsible federation would require, but none has been exercised with real contributors, consent decisions, or a running federation service.

Rather than claiming a validated citizen observatory, this paper contributes the node-level architecture and deployment framework required to enable a future multi-node participatory evaluation under realistic maritime conditions.

The central systems insight is that end-to-end embedded AI performance cannot be represented by inference latency alone: in the initial implementation, persistence and request coordination dominated avoidable latency, and stage-level measurement enabled a 72% improvement without replacing the model – directly relevant to citizen-science deployments, where local processing and synchronization policies affect responsiveness and contributor experience.

B. Design Lessons for Citizen-Science Infrastructure

Three lessons emerge from the prototype. First, a common replay path lets protocol, detector, and persistence changes be evaluated on retained observations before they affect contributors. Second, health and provenance must be first-class outputs: a visually plausible frame is scientifically weak when device state or timing context is unknown. Third, local autonomy and governance are compatible: edge processing can reduce transmission and support immediate feedback, while contributor review and selective synchronization limit unnecessary disclosure.

C. Validity Considerations

Internal validity. The experiments were conducted on one Raspberry Pi 4 under controlled conditions. The fifty-request inference sample is descriptive of this device and commit. Continuous inference and synthetic thermal stress were exercised as separate experiments (Section V-D), not folded into this percentile sample, and repeated thermal acquisition remains untested. OS scheduling, cache state, and local-network effects may influence the measurements.

Construct validity. The evaluation concerns software integration, repeatability, resource demand, and request-level timing; it does not establish detector accuracy, calibrated uncertainty, RGB-thermal fusion, or safety performance. The node-to-observatory contract is a design contribution, not empirical evidence of engagement or societal impact.

External validity. Generalization to other devices, sensor combinations, and multi-day deployments is not yet established; the two-hour run (Section V-D) evidences short-duration stability only, and the thermal result is specific to this unit’s passive cooling and ambient conditions. The node assumes only a local-area connection – acquisition, inference, and persistence need no internet access, only the dashboard needs a LAN link – consistent

with the intermittent wide-area connectivity anticipated for federation (Section IV-C); a degraded wide-area link to a remote observatory has not been measured. Future work will address multi-node replication, in-line power measurement, intermittent connectivity, storage pressure, fault recovery, labelled field data, and contributor studies.

VII. Reproducibility and Roadmap

Internal repeatability is supported by the common live/replay abstraction, normalized detections, explicit state, and stage timing. Independent replication requires a versioned artifact with source code, dependency versions, model checksum, configuration, measurement scripts, and representative replay inputs, forming the planned reproducibility package.

A. Artifact Availability

The implementation is available through two repositories: `easy-maritime-awareness`¹ for dataset preparation and the exported ONNX model, and `EASY-Maritime-Awareness-Dashboard`² for the integrated Raspberry Pi runtime, REST interface, and benchmark tooling. A companion archive is planned to preserve raw benchmark data and inputs needed to reproduce the reported tables.

Future work will federate multiple nodes operated by volunteer operators, evaluating enrollment, secure synchronization, privacy-preserving dissemination, usability, and consent comprehension, through a laboratory replication campaign preceding a supervised pilot and broader field study. On the perception side, labelled maritime recordings will be evaluated across object classes, illumination, and sea states, with RGB-thermal fusion following only after timestamp and geometric calibration. This evaluation already characterized thermal and undervoltage margin under load (Section V-D); planned instrumentation adds an in-line power meter (e.g., INA219/INA226) alongside multi-day operation, intermittent connectivity, and fault-injection experiments.

VIII. Conclusion

The proposed system is a modular, measurable edge node for citizen-contributed maritime observations, with stable boundaries among devices, sources, frame delivery, inference, detections, events, sessions, persistence, and interfaces exposing live and replay inputs through common downstream contracts. The prototype sustained two 10-FPS RGB feeds with modest resource use, held that envelope under thermal stress and two hours of continuous inference, and request-path instrumentation substantially

reduced non-model latency. The same instrumentation also exposed the two boundaries of the current build: a thin thermal margin under full load, and request latency that doubles when the node is driven continuously rather

¹<https://github.com/carmincoppola/easy-maritime-awareness>

²<https://github.com/carmincoppola/EASY-Maritime-Awareness-Dashboard>
than on demand. The platform should be regarded as a validated edge node and deployment framework, not a completed citizen observatory, providing the software and governance foundation for a future multi-node field study.

Acknowledgements

This work is partially funded by the EASY project (CUP I63C25000510007).

References

- [1] R. Bonney, C. B. Cooper, J. Dickinson, S. Kelling, T. Phillips, K. V. Rosenberg, and J. Shirk, "Citizen science: A developing tool for expanding science knowledge and scientific literacy," *BioScience*, vol. 59, no. 11, pp. 977–984, 2009.
- [2] F. M. Lauro, J. J. Senstius, J. Cullen, R. Neches, R. M. Jensen, M. V. Brown, A. E. Darling, M. Givskov, D. McDougald, R. Hoeke, M. Ostrowski, G. K. Philip, I. T. Paulsen, and J. J. Grzyski, "The common oceanographer: Crowdsourcing the collection of oceanographic data," *PLoS Biology*, vol. 12, no. 9, p. e1001947, 2014.
- [3] M. Kosmala, A. Wiggins, A. Swanson, and B. Simmons, "Assessing data quality in citizen science," *Frontiers in Ecology and the Environment*, vol. 14, no. 10, pp. 551–560, 2016.
- [4] A. Grainger, "Citizen observatories and the new earth observation science," *Remote Sensing*, vol. 9, no. 2, p. 153, 2017.
- [5] U. Wehn, M. Gharesifard, L. Ceccaroni, H. Joyce, R. Ajates, S. Woods, A. Bilbao, S. Parkinson, M. Gold, and J. Wheatland, "Impact assessment of citizen science: State of the art and guiding principles for a consolidated approach," *Sustainability Science*, vol. 16, pp. 1683–1699, 2021.
- [6] C. Kullenberg and D. Kasperowski, "What is citizen science?—a scientometric meta-analysis," *PLoS ONE*, vol. 11, no. 1, p. e0147152, 2016.
- [7] L. A. Varga, B. Kiefer, M. Messmer, and A. Zell, "Seadronessee: A maritime benchmark for detecting humans in open water," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 2260–2270.
- [8] M. Lacki, "Determining the level of threat in maritime navigation based on the detection of small floating objects with deep neural networks," *Sensors*, vol. 24, no. 23, p. 7505, 2024.
- [9] J.-H. Kim, N. Kim, Y.-W. Park, and C.-S. Won, "Object detection and classification based on YOLO-v5 with improved maritime dataset," *Journal of Marine Science and Engineering*, vol. 10, no. 3, p. 377, 2022.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [11] Z. Shao, W. Wu, Z. Wang, W. Du, and C. Li, "SeaShips: A large-scale precisely annotated dataset for ship detection," *IEEE Transactions on Multimedia*, vol. 20, no. 10, pp. 2593–2604, 2018.